📘 Django Interview Notes (Day 1)

🧠 1. What is Django?

Django is a high-level Python web framework used to build secure, scalable, and maintainable web applications quickly. It follows the MVT architecture and provides built-in features like authentication, ORM, admin panel, and security protections.

🏗️ 2. What is MVT Architecture?

MVT stands for Model, View, Template:

- **Model** → Defines database structure (tables, fields)
- **View** → Contains business logic and handles request/response
- **Template** → Handles UI (HTML, CSS, JS)

👉 In backend APIs, templates are usually replaced with JSON responses.

🔄 3. Request Flow in Django

When a request hits Django:

1. Request goes to urls.py
2. URL pattern is matched
3. Request is forwarded to View
4. View processes logic
5. View interacts with Model (DB)
6. Data returns to View
7. Response (JSON/HTML) is sent back to client

⚙️ 4. What is manage.py?

manage.py is a command-line utility used to interact with Django projects.

It is used for:

- Running server
- Creating apps
- Applying migrations
- Running commands

🧩 5. What is a Django App?

A Django app is a modular component that handles a specific functionality.

Example:

- users → authentication
- products → product management
- orders → order system

👉 A project is made up of multiple apps.

🔗 6. What is urls.py and include()?

- urls.py is used for routing requests to appropriate views.
- include() is used to connect app-level URLs to project-level URLs.

👉 It helps in modular routing.

🏢 7. Difference Between Project and App

Project	App
Complete application	Small module
Contains settings	Contains logic
Can have multiple apps	Focused functionality

⚡ 8. WSGI vs ASGI

WSGI	ASGI
Synchronous	Asynchronous
Traditional apps	Real-time apps
No WebSocket support	Supports WebSockets

🔋 9. Why Django is called “Batteries Included”?

Because Django provides built-in features:

- Authentication

- Admin panel
- ORM
- Security

👉 No need to build from scratch.

⚔️ 10. Django vs FastAPI

Django	FastAPI
Full framework	Lightweight
Built-in features	Minimal
Slower	Faster
Good for large apps	Good for microservices

👉 Use FastAPI when performance & async are critical.

🧪 Standard API Response Format

```
{
  "status": "success",
  "message": "User fetched successfully",
  "data": {}
}
```

📌 Key Takeaways

- Django follows MVT architecture
- Request flow is very important (interview favorite)
- Apps make code modular
- Django provides many built-in features
- Understand difference between sync and async